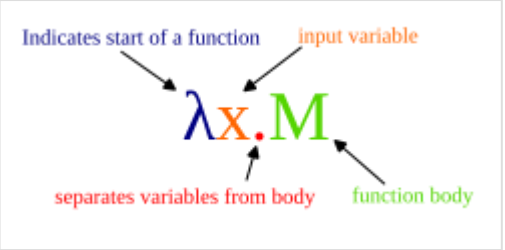
Lambda calculus

In mathematical logic, the **lambda calculus** (also written as **λ-calculus**) is a formal system for expressing computation based on function abstraction and application using variable binding and substitution. Untyped lambda calculus, the topic of this article, is a universal machine, i.e. a model of computation that can be used to simulate any Turing machine (and vice versa). It was introduced by the mathematician Alonzo Church in the 1930s as part of his research into the foundations of mathematics. In 1936, Church found a formulation which was logically consistent, and documented it in 1940.



The lambda abstraction decomposed. The λ indicates the start of a function. x is the input parameter. M is the body, separated by a dot separator "." from the input parameter.

Definition

The lambda calculus consists of a language of **lambda terms**, that are defined by a formal syntax, and a set of transformation rules for manipulating those terms. In BNF, the syntax is $e ::= x \mid \lambda x.\, e \mid e\, e$, where variables x,y,z range over an infinite set of names. Terms M,N,t,s,e,f range over all lambda terms. This corresponds to the following inductive definition:

- A *variable* x is a valid lambda term.
- An *abstraction* is a lambda term $(\lambda x.\, t)$ where t is a lambda term, referred to as the abstraction's *body*, and x is the abstraction's *parameter* variable,
- An *application* is a lambda term $(t\, s)$ where t and s are lambda terms.

A lambda term is syntactically valid if it can be obtained by repeated application of these three rules. For convenience, parentheses can often be omitted when writing a lambda term—see Lambda calculus definition § Notation for details.

Within lambda terms, any occurrence of a variable that is not a parameter of some enclosing λ is said to be *free*. Any free occurrence of x in a term M is *bound* in $\lambda x.\, M$. Any free occurrence of any other variable within M remains free in $\lambda x.\, M$.

For example, in the term $x\, y$, both x and y occur free. In $(\lambda x.\, x\, y)$, y is free, but x in the body (i.e. after the dot) is not free, and is said to be *bound* (to the parameter). While y is free in $(\lambda x.\, x\, y)$, it is bound in $(\lambda y.\, \lambda x.\, x\, y)$. There are two occurrences of x in $(\lambda y.\, (\lambda x.\, x\, y)\, x)$ – one is bound, and the other is free.

FV(M) is the set of free variables of M, i.e. such variables that occur free in M at least once. It can be defined inductively as follows:

- $\mathbf{FV}(x) = \{x\}$
- $\mathbf{FV}(M_1 M_2) = \mathbf{FV}(M_1) \cup \mathbf{FV}(M_2)$
- $\mathbf{FV}(\lambda x.\, M) = \mathbf{FV}(M) \backslash \{x\}$

The notation $M[x := N]$ denotes **capture-avoiding substitution**: substituting N for every free occurrence of x in M, while avoiding variable capture.[a] This operation is defined inductively as follows:

- $x[x := N] = N$; $y[x := N] = y$ if $y \neq x$.
- $(M_1 M_2)[x := N] = (M_1[x := N])(M_2[x := N])$.
- $(\lambda y.\, M)[x := N]$ has three cases:
  - If $y = x$, becomes $\lambda x.\, M$ (x is bound; no change).
  - If $y \notin FV(N)$, becomes $\lambda y.\, (M[x := N])$.
  - If $y \in FV(N)$, first α-rename $\lambda y.\, M$ to $\lambda y'.\, M[y := y']$ with y' fresh to avoid name collisions, then continue as above. It becomes $\lambda y'.\, M[y := y'][x := N]$with $y' \notin \mathrm{FV}(M) \cup \mathrm{FV}(N)$.

There are several notions of "equivalence" and "reduction" that make it possible to reduce lambda terms to equivalent lambda terms.[3]

- *α-conversion* captures the intuition that the particular choice of a bound variable, in an abstraction, does not (usually) matter. If $y \notin FV(M)$, then the terms $\lambda x.\, M$ and $\lambda y.\, M[x := y]$ are considered *alpha-equivalent*, written $\lambda x.\, M \equiv_\alpha \lambda y.\, M[x := y]$. The equivalence relation is the smallest congruence relation on lambda terms generated by this rule. For instance, $\lambda x.\, x$ and $\lambda y.\, y$ are alpha-equivalent lambda terms.
- The *β-reduction* rule states that a β-redex, an application of the form $(\lambda x.\, t)s$, reduces to the term $t[x := s]$.[b] For example, for every s, $(\lambda x.\, x)s \to x[x := s] = s$. This demonstrates that $\lambda x.\, x$ really is the identity. Similarly, $(\lambda x.\, y)s \to y[x := s] = y$, which demonstrates that $\lambda x.\, y$ is a constant function.
- *η-conversion* expresses extensionality and converts between $\lambda x.\, fx$ and f whenever x does not appear free in f. It is often omitted in many treatments of lambda calculus.

The term *redex*, short for *reducible expression*, refers to subterms that can be reduced by one of the reduction rules. For example, (λ*x*.*M*) *N* is a β-redex in expressing the substitution of *N* for *x* in *M*. The expression to which a redex reduces is called its *reduct*; the reduct of (λ*x*.*M*) *N* is *M*[*x* := *N*].

Explanation and applications

Lambda calculus is Turing complete, that is, it is a universal model of computation that can be used to simulate any Turing machine.[4] Its namesake, the Greek letter lambda (λ), is used in lambda expressions and lambda terms to denote binding a variable in a function.

Lambda calculus may be *untyped* or *typed*. In typed lambda calculus, functions can be applied only if they are capable of accepting the given input's "type" of data. Typed lambda calculi are strictly *weaker* than the untyped lambda calculus, which is the primary subject of this article, in the sense that *typed lambda calculi can express less* than the untyped calculus can. On the other hand, more things can be proven with typed lambda calculi. For example, in simply typed lambda calculus, it is a theorem that every evaluation strategy terminates for every simply typed lambda-term,[5] whereas evaluation of untyped lambda-terms need not terminate (see below). One reason there are many different typed lambda calculi has been the desire to do more (of what the untyped calculus can do) without giving up on being able to prove strong theorems about the calculus.

Lambda calculus has applications in many different areas in mathematics, philosophy,[6] linguistics,[7][8] and computer science.[9][10] Lambda calculus has played an important role in the development of the theory of programming languages. Functional programming languages implement lambda calculus. Lambda calculus is also a current research topic in category theory.[11]

History

Lambda calculus was introduced by mathematician Alonzo Church in the 1930s as part of an investigation into the foundations of mathematics.[12][c] The original system was shown to be logically inconsistent in 1935 when Stephen Kleene and J. B. Rosser developed the Kleene–Rosser paradox.[13][14]

Subsequently, in 1936 Church isolated and published just the portion relevant to computation, what is now called the untyped lambda calculus.[15] In 1940, he also introduced a computationally weaker, but logically consistent system, known as the simply typed lambda calculus.[16]

Until the 1960s when its relation to programming languages was clarified, the lambda calculus was only a formalism. Thanks to Richard Montague and other linguists' applications in the semantics of natural language, the lambda calculus has begun to enjoy a respectable place in both linguistics[17] and computer science.[18]

Origin of the *λ* symbol

There is some uncertainty over the reason for Church's use of the Greek letter lambda (λ) as the notation for function-abstraction in the lambda calculus, perhaps in part due to conflicting explanations by Church himself. According to Cardone and Hindley (2006):

> By the way, why did Church choose the notation "λ"? In [an unpublished 1964 letter to Harald Dickson] he stated clearly that it came from the notation "$\hat{x}$" used for class-abstraction by Whitehead and Russell, by first modifying "$\hat{x}$" to "$\wedge x$" to distinguish function-abstraction from class-abstraction, and then changing "∧" to "λ" for ease of printing.
>
> This origin was also reported in [Rosser, 1984, p.338]. On the other hand, in his later years Church told two enquirers that the choice was more accidental: a symbol was needed and λ just happened to be chosen.

Dana Scott has also addressed this question in various public lectures.[19] Scott recounts that he once posed a question about the origin of the lambda symbol to Church's former student and son-in-law John W. Addison Jr., who then wrote his father-in-law a postcard:

> Dear Professor Church,
>
> Russell had the iota operator, Hilbert had the epsilon operator. Why did you choose lambda for your operator?

According to Scott, Church's entire response consisted of returning the postcard with the following annotation: "eeny, meeny, miny, moe".

Motivation

Computable functions are a fundamental concept within computer science and mathematics. The lambda calculus provides simple semantics for computation which are useful for formally studying properties of computation. The lambda calculus incorporates two simplifications that make its semantics simple. The first simplification is that the lambda calculus treats functions "anonymously"; it does not give them explicit names. For example, the function

$$\mathbf{square\_sum}(x, y) = x^2 + y^2$$

can be rewritten in *anonymous form* as

$$(x, y) \mapsto x^2 + y^2$$

(which is read as "a tuple of x and y is mapped to $x^2 + y^2$").[d] Similarly, the function

$$\mathrm{id}(x) = x$$

can be rewritten in anonymous form as

$$x \mapsto x$$

where the input is simply mapped to itself.[d]

The second simplification is that the lambda calculus only uses functions of a single input. An ordinary function that requires two inputs, for instance the **square_sum** function, can be reworked into an equivalent function that accepts a single input, and as output returns *another* function, that in turn accepts a single input. For example,

$$(x, y) \mapsto x^2 + y^2$$

can be reworked into

$$x \mapsto (y \mapsto x^2 + y^2)$$

This method, known as currying, transforms a function that takes multiple arguments into a chain of functions each with a single argument.

Function application of the **square_sum** function to the arguments (5, 2), yields at once

$$\begin{aligned}&((x, y) \mapsto x^2 + y^2)(5, 2)\\&= 5^2 + 2^2\\&= 29,\end{aligned}$$

whereas evaluation of the curried version requires one more step

$\Big((x \mapsto (y \mapsto x^2 + y^2))(5)\Big)(2)$
$= (y \mapsto 5^2 + y^2)(2)$ // the definition of x has been used with 5 in the inner expression. This is like β-reduction.
$= 5^2 + 2^2$ // the definition of y has been used with 2. Again, similar to β-reduction.
$= 29$

to arrive at the same result.

In lambda calculus, functions are taken to be 'first class values', so functions may be used as the inputs, or be returned as outputs from other functions. For example, the lambda term $\lambda x.\, x$ represents the identity function, $x \mapsto x$. Further, $\lambda x.\, y$ represents the *constant function* $x \mapsto y$, the function that always returns y, no matter the input. As an example of a function operating on functions, the function composition can be defined as $\lambda f.\, \lambda g.\, \lambda x.\, (f(gx))$.

Normal forms and confluence

It can be shown that β-reduction is confluent when working up to α-conversion (i.e. we consider two normal forms to be equal if it is possible to α-convert one into the other). If repeated application of the reduction steps eventually terminates, then by the Church–Rosser theorem it will produce a unique β-normal form. However, the untyped lambda calculus as a rewriting rule under β-reduction is neither strongly normalising nor weakly normalising; there are terms with no normal form such as $\mathbf{\Omega}$.

Considering individual terms, both strongly normalising terms and weakly normalising terms have a unique normal form. For strongly normalising terms, any reduction strategy is guaranteed to yield the normal form, whereas for weakly normalising terms, some reduction strategies may fail to find it.

Encoding datatypes

The basic lambda calculus may be used to model arithmetic, Booleans, data structures, and recursion, as illustrated in the following sub-sections *i*, *ii*, *iii*, and *§ iv*.

Arithmetic in lambda calculus

There are several possible ways to define the natural numbers in lambda calculus, but by far the most common are the Church numerals, which can be defined as follows:

```
0 := λf.λx.x
1 := λf.λx.f x
2 := λf.λx.f (f x)
3 := λf.λx.f (f (f x))
```

and so on. Or using an alternative syntax allowing multiple uncurried arguments to a function:

```
0 := λfx.x
1 := λfx.f x
2 := λfx.f (f x)
```

```
3 := λfx.f (f (f x))
```

A Church numeral is a higher-order function—it takes a single-argument function f, and returns another single-argument function. The Church numeral n is a function that takes a function f as argument and returns the n-th composition of f, i.e. the function f composed with itself n times. This is denoted $f^{(n)}$ and is in fact the n-th power of f (considered as an operator); $f^{(0)}$ is defined to be the identity function. Functional composition is associative, and so, such repeated compositions of a single function f obey two laws of exponents, $f^{(m)} \circ f^{(n)} = f^{(m+n)}$ and $(f^{(n)})^{(m)} = f^{(m*n)}$, which is why these numerals can be used for arithmetic. (In Church's original lambda calculus, the formal parameter of a lambda expression was required to occur at least once in the function body, which made the above definition of `0` impossible.)

One way of thinking about the Church numeral n, which is often useful when analyzing programs, is as an instruction 'repeat n times'. For example, using the `PAIR` and `NIL` functions defined below, one can define a function that constructs a (linked) list of n elements all equal to x by repeating 'prepend another x element' n times, starting from an empty list. The lambda term

```
λn.λx.n (PAIR x) NIL
```

creates, given a Church numeral n and some x, a sequence of n applications

```
PAIR x (PAIR x...(PAIR x NIL)...)
```

By varying what is being repeated, and what argument(s) that function being repeated is applied to, a great many different effects can be achieved.

We can define a successor function, which takes a Church numeral n and returns its successor $n + 1$ by performing one additional application of the function f it is supplied with, where $(n\ f\ x)$ means "n applications of f starting from x":

```
SUCC := λn.λf.λx.f (n f x)
```

Because the m-th composition of f composed with the n-th composition of f gives the $m+n$-th composition of f, $f^{(m)} \circ f^{(n)} = f^{(m+n)}$, addition can be defined as

```
PLUS := λm.λn.λf.λx.m f (n f x)
```

`PLUS` can be thought of as a function taking two natural numbers as arguments and returning a natural number; it can be verified that

```
PLUS 2 3
```

and

```
5
```

are beta-equivalent lambda expressions. Since adding m to a number can be accomplished by repeating the successor operation m times, an alternative definition is:

```
PLUS′ := λm.λn.m SUCC n
```
[20]

Similarly, following $(f^{(n)})^{(m)} = f^{(m*n)}$, multiplication can be defined as

$$\mathsf{MULT} := \lambda m.\lambda n.\lambda f.m\ (n\ f)$$ [21]

Thus multiplication of Church numerals is simply their composition as functions. Alternatively

$$\mathsf{MULT'} := \lambda m.\lambda n.m\ (\mathsf{PLUS}\ n)\ 0$$

since multiplying m and n is the same as adding n repeatedly, m times, starting from zero.

Exponentiation, being the repeated multiplication of a number with itself, translates as a repeated composition of a Church numeral with itself, as a function. And repeated composition is what Church numerals *are*:

$$\mathsf{POW} := \lambda b.\lambda n.n\ b$$ [1]

Alternatively here as well,

$$\mathsf{POW'} := \lambda b.\lambda n.n\ (\mathsf{MULT}\ b)\ 1$$

Simplifying, it becomes

$$\mathsf{POW''} := \lambda b.\lambda n.\lambda f.n\ b\ f$$

but that is just an eta-expanded version of POW we already have, above.

The *predecessor* function, specified by two equations $\mathsf{PRED}\ (\mathsf{SUCC}\ n) = n$ and $\mathsf{PRED}\ 0 = 0$, is considerably more involved. The formula

$$\mathsf{PRED} := \lambda n.\lambda f.\lambda x.n\ (\lambda g.\lambda h.h\ (g\ f))\ (\lambda u.x)\ (\lambda u.u)$$

can be validated by showing inductively that if T denotes $(\lambda g.\lambda h.h\ (g\ f))$, then $\mathsf{T}^{(n)}(\lambda u.x) = (\lambda h.h(f^{(n-1)}(x)))$ for $n > 0$. Two other definitions of PRED are given below, one using conditionals and the other using pairs. With the predecessor function, subtraction is straightforward. Defining

$$\mathsf{SUB} := \lambda m.\lambda n.n\ \mathsf{PRED}\ m,$$

$\mathsf{SUB}\ m\ n$ yields $m - n$ when $m > n$ and 0 otherwise.

Logic and predicates

By convention, the following two definitions (known as Church Booleans) are used for the Boolean values TRUE and FALSE:

$$\mathsf{TRUE} := \lambda x.\lambda y.x$$
$$\mathsf{FALSE} := \lambda x.\lambda y.y$$

Then, with these two lambda terms, we can define some logic operators (these are just possible formulations; other expressions could be equally correct):

```
AND := λp.λq.p q p
OR := λp.λq.p p q
NOT := λp.p FALSE TRUE
IFTHENELSE := λp.λa.λb.p a b
```

We are now able to compute some logic functions, for example:

```
AND TRUE FALSE
     ≡ (λp.λq.p q p) TRUE FALSE →β TRUE FALSE TRUE
     ≡ (λx.λy.x) FALSE TRUE →β FALSE
```

and we see that `AND TRUE FALSE` is equivalent to `FALSE`.

A *predicate* is a function that returns a Boolean value. The most fundamental predicate is `ISZERO`, which returns `TRUE` if its argument is the Church numeral `0`, but `FALSE` if its argument were any other Church numeral:

```
ISZERO := λn.n (λx.FALSE) TRUE
```

The following predicate tests whether the first argument is less-than-or-equal-to the second:

```
LEQ := λm.λn.ISZERO (SUB m n),
```

and since $m = n$, if `LEQ m n` and `LEQ n m`, it is straightforward to build a predicate for numerical equality.

The availability of predicates and the above definition of `TRUE` and `FALSE` make it convenient to write "if-then-else" expressions in lambda calculus. For example, the predecessor function can be defined as:

```
PRED := λn.n (λg.λk.ISZERO (g 1) k (PLUS (g k) 1)) (λv.0) 0
```

which can be verified by showing inductively that `n (λg.λk.ISZERO (g 1) k (PLUS (g k) 1)) (λv.0)` is the add $n - 1$ function for $n > 0$.

Pairs

A pair (2-tuple) encapsulates two values, and is represented by an abstraction that expects a handler to which it will pass the two values. `FIRST` returns the first element of the pair, and `SECOND` returns the second.

```
PAIR := λx.λy.λf.f x y
FIRST := λp.p (λx.λy.x)
SECOND := λp.p (λx.λy.y)
```

A linked list can either be NIL, representing the empty list, or a `PAIR` of an element (so-called *head*) and a smaller list (*tail*). The predicate `NULL` returns `TRUE` for the value `NIL`, and `FALSE` for a non-empty list:

```
NIL := λf.TRUE
NULL := λp.p (λx.λy.FALSE)
```

Alternatively, with NIL := FALSE, the construct (*l* (λ*h*.λ*t*.λ*z*. ...*h*...*t*...) _on_nil_) obviates the need for an explicit NULL test:

```
NIL := λx.λy.y
NULL := λl.l (λh.λt.λz.FALSE) TRUE
```

As an example of the use of pairs, the shift-and-increment function that maps (*m*, *n*) to (*n*, *n* + 1) can be defined as

```
Φ := λp.PAIR (SECOND p) (SUCC (SECOND p))
Ψ := λfp.PAIR (SECOND p) (f (SECOND p))
```

which allows us to give perhaps the most transparent version of the predecessor function:

```
PRED := λn.FIRST (n (Ψ SUCC) (PAIR 0 0))
  = λnfx.FIRST (n (Ψ f) (PAIR x x))
```

Substituting the definitions and simplifying the resulting expression leads to streamlined definitions

```
  = λnfx.n (λrab.rb(fb)) (λab.a) x x
  = λnfx.n (λrij.j(rjf)) (λij.x) I I
  = λnfx.n (λrij.i(rjj)) (λij.x) I f
  = λnfx.n (λri.i(rf)) (λi.x) I
```

(where I := λ*x*.*x*), evidently leading back to the original.

Additional programming techniques

There is a considerable body of programming idioms for lambda calculus. Many of these were originally developed in the context of using lambda calculus as a foundation for programming language semantics, effectively using lambda calculus as a low-level programming language. Because several programming languages include the lambda calculus (or something very similar) as a fragment, these techniques also see use in practical programming, but may then be perceived as obscure or foreign.

Named constants

In lambda calculus, a library would take the form of a collection of previously defined functions, which as lambda-terms are merely particular constants. The pure lambda calculus does not have a concept of named constants since all atomic lambda-terms are variables, but one can emulate having named constants by setting aside a variable as the name of the constant, using abstraction to bind that variable in the main body, and apply that abstraction to the intended definition. Thus to use *f* to mean *N* (some explicit lambda-term) in *M* (another lambda-term, the "main program"), one can say

```
(λf.M) N
```

Authors often introduce syntactic sugar, such as let,[e] to permit writing the above in the more intuitive order

```
let f = N in M
```

By chaining such definitions, one can write a lambda calculus "program" as zero or more function definitions, followed by one lambda-term using those functions that constitutes the main body of the program.

A notable restriction of this `let` is that the name *f* may not be referenced in *N*, for *N* is outside the scope of the abstraction binding *f*, which is *M*; this means a recursive function definition cannot be written with `let`. The `letrec`[f] construction would allow writing recursive function definitions, where the scope of the abstraction binding *f* includes *N* as well as *M*. Or self-application a-la that which leads to **Y** combinator could be used.

Recursion and fixed points

Recursion is when a function invokes itself. What would a value be which were to represent such a function? It has to refer to itself somehow inside itself, just as the definition refers to itself inside itself. If this value were to contain itself by value, it would have to be of infinite size, which is impossible. Other notations, which support recursion natively, overcome this by referring to the function *by name* inside its definition. Lambda calculus cannot express this, since in it there simply are no names for terms to begin with, only arguments' names, i.e. parameters in abstractions. Thus, a lambda expression can receive itself as its argument and refer to (a copy of) itself via the corresponding parameter's name. This will work fine in case it was indeed called with itself as an argument. For example, $(\lambda x.\, x\ x)\ E = (E\ E)$ will express recursion when *E* is an abstraction which is applying its parameter to itself inside its body to express a recursive call. Since this parameter receives *E* as its value, its self-application will be the same $(E\ E)$ again.

As a concrete example, consider the factorial function $F(n)$, recursively defined by

$$F(n) = 1,\ \text{if } n = 0;\ \text{else } n \times F(n-1).$$

In the lambda expression which is to represent this function, a *parameter* (typically the first one) will be assumed to receive the lambda expression itself as its value, so that calling it with itself as its first argument will amount to the recursive call. Thus to achieve recursion, the intended-as-self-referencing argument (called *s* here, reminiscent of "self", or "self-applying") must always be passed to itself within the function body at a recursive call point:

$$E := \lambda s.\ \lambda n.(1,\ \text{if } n = 0;\ \text{else } n \times (s\ s\ (n-1)))$$

with $s\ s\ n = F\ n = E\ E\ n$ to hold, so $s = E$ and

$$F := (\lambda x.\, x\ x)\ E = E\ E$$

and we have

$$F = E\ E = \lambda n.(1,\ \text{if } n = 0;\ \text{else } n \times (E\ E\ (n-1)))$$

Here $s\ s$ becomes *the same* $(E\ E)$ inside the result of the application $(E\ E)$, and using the same function for a call is the definition of what recursion is. The self-application achieves replication here, passing the function's lambda expression on to the next invocation as an argument value, making it available to be referenced there by the parameter name *s* to be called via the self-application $s\ s$, again and again as needed, each time *re-creating* the lambda-term $F = E\ E$.

The application is an additional step just as the name lookup would be. It has the same delaying effect. Instead of having F inside itself as a whole *up-front*, delaying its re-creation until the next call makes its existence possible by having two *finite* lambda-terms E inside it re-create it on the fly *later* as needed.

This self-applicational approach solves it, but requires re-writing each recursive call as a self-application. We would like to have a generic solution, without the need for any re-writes:

```
G := λr. λn.(1, if n = 0; else n × (r (n−1)))
        with r x = F x = G r x to hold, so r = G r =: FIX G and
F := FIX G where FIX g = (r where r = g r) = g (FIX g)
        so that FIX G = G (FIX G) = (λn.(1, if n = 0; else n × ((FIX
        G) (n−1))))
```

Given a lambda term with first argument representing recursive call (e.g. G here), the *fixed-point* combinator FIX will return a self-replicating lambda expression representing the recursive function (here, F). The function does not need to be explicitly passed to itself at any point, for the self-replication is arranged in advance, when it is created, to be done each time it is called. Thus the original lambda expression (FIX G) is re-created inside itself, at call-point, achieving self-reference.

In fact, there are many possible definitions for this FIX operator, the simplest of them being:

```
Y := λg.(λx.g (x x)) (λx.g (x x))
```

In the lambda calculus, **Y** *g* is a fixed-point of *g*, as it expands to:

```
Y g
~> (λh.(λx.h (x x)) (λx.h (x x))) g
~> (λx.g (x x)) (λx.g (x x))
~> g ((λx.g (x x)) (λx.g (x x)))
<~ g (Y g)
```

Now, to perform the recursive call to the factorial function for an argument *n*, we would simply call (**Y** G) *n*. Given *n* = 4, for example, this gives:

```
(Y G) 4
~> G (Y G) 4
~> (λr.λn.(1, if n = 0; else n × (r (n−1)))) (Y G) 4
~> (λn.(1, if n = 0; else n × ((Y G) (n−1)))) 4
~> 1, if 4 = 0; else 4 × ((Y G) (4−1))
~> 4 × (G (Y G) (4−1))
~> 4 × ((λn.(1, if n = 0; else n × ((Y G) (n−1)))) (4−1))
~> 4 × (1, if 3 = 0; else 3 × ((Y G) (3−1)))
~> 4 × (3 × (G (Y G) (3−1)))
~> 4 × (3 × ((λn.(1, if n = 0; else n × ((Y G) (n−1)))) (3−1)))
~> 4 × (3 × (1, if 2 = 0; else 2 × ((Y G) (2−1))))
~> 4 × (3 × (2 × (G (Y G) (2−1))))
~> 4 × (3 × (2 × ((λn.(1, if n = 0; else n × ((Y G) (n−1)))) (2−1))))
~> 4 × (3 × (2 × (1, if 1 = 0; else 1 × ((Y G) (1−1)))))
~> 4 × (3 × (2 × (1 × (G (Y G) (1−1)))))
~> 4 × (3 × (2 × (1 × ((λn.(1, if n = 0; else n × ((Y G) (n−1)))) (1−1)))))
```

```
~> 4 × (3 × (2 × (1 × (1, if 0 = 0; else 0 × ((Y G) (0−1))))))
~> 4 × (3 × (2 × (1 × (1))))
~> 24
```

Every recursively defined function can be seen as a fixed point of some suitably defined higher order function (also known as functional) closing over the recursive call with an extra argument. Therefore, using **Y**, every recursive function can be expressed as a lambda expression. In particular, we can now cleanly define the subtraction, multiplication, and comparison predicates of natural numbers, using recursion.

When Y combinator is coded directly in a strict programming language, the applicative order of evaluation used in such languages will cause an attempt to fully expand the internal self-application (xx) prematurely, causing stack overflow or, in case of tail call optimization, indefinite looping.[23] A delayed variant of Y, the Z combinator, can be used in such languages. It has the internal self-application hidden behind an extra abstraction through eta-expansion, as $(\lambda v.\, xxv)$, thus preventing its premature expansion:[24]

$$Z = \lambda f.\, (\lambda x.\, f(\lambda v.\, xxv))\ (\lambda x.\, f(\lambda v.\, xxv))\ .$$

Standard terms

Certain terms have commonly accepted names:[25][26][27]

```
I := λx.x
S := λx.λy.λz.x z (y z)
K := λx.λy.x
B := λx.λy.λz.x (y z)
C := λx.λy.λz.x z y
W := λx.λy.x y y
ω or Δ or U := λx.x x
Ω := ω ω
```

I is the identity function. **SK** and **BCKW** form complete combinator calculus systems that can express any lambda term - see the next section. **Ω** is **UU**, the smallest term that has no normal form. **YI** is another such term. **Y** is standard and defined above, and can also be defined as **Y=BU(CBU)**, so that **Y**g=g(**Y**g). TRUE and FALSE defined above are commonly abbreviated as **T** and **F**.

Abstraction elimination

If *N* is a lambda-term without abstraction, but possibly containing named constants (combinators), then there exists a lambda-term *T*(*x*,*N*) which is equivalent to $\lambda x.\, N$ but lacks abstraction (except as part of the named constants, if these are considered non-atomic). This can also be viewed as anonymising variables, as *T*(*x*,*N*) removes all occurrences of *x* from *N*, while still allowing argument values to be substituted into the positions where *N* contains an *x*. The conversion function *T* can be defined by:

T(*x*, *x*) := **I**
T(*x*, *N*) := **K** *N* if *x* is not free in *N*.
T(*x*, *M N*) := **S** *T*(*x*, *M*) *T*(*x*, *N*)

In either case, a term of the form $T(x,N)\ P$ is reduced by having the initial combinator **I**, **K**, or **S** grab the argument P, just like β-reduction of $(\lambda x.N)\ P$ would do. **I** returns that argument. **K** N throws the argument away, just like $(\lambda x.N)$ does when x has no free occurrence in N. **S** passes the argument on to both subterms of the application, and then applies the result of the first to the result of the second, just like $(\lambda x.MN)P$ is the same as $((\lambda x.M)P)\ ((\lambda x.N)P)$.

The combinators **B** and **C** are similar to **S**, but pass the argument on to only one subterm of an application (**B** to the "argument" subterm and **C** to the "function" subterm), thus saving a subsequent **K** if there is no occurrence of x in one subterm. In comparison to **B** and **C**, the **S** combinator actually conflates two functionalities: rearranging arguments, and duplicating an argument so that it may be used in two places. The **W** combinator does only the latter, yielding the B, C, K, W system as an alternative to SKI combinator calculus.

Typed lambda calculus

A *typed lambda calculus* is a typed formalism that uses the lambda-symbol (λ) to denote anonymous function abstraction. In this context, types are usually objects of a syntactic nature that are assigned to lambda terms; the exact nature of a type depends on the calculus considered (see Kinds of typed lambda calculi). From a certain point of view, typed lambda calculi can be seen as refinements of the untyped lambda calculus but from another point of view, they can also be considered the more fundamental theory and *untyped lambda calculus* a special case with only one type.[28]

Typed lambda calculi are foundational programming languages and are the base of typed functional programming languages such as ML and Haskell and, more indirectly, typed imperative programming languages. Typed lambda calculi play an important role in the design of type systems for programming languages; here typability usually captures desirable properties of the program, e.g., the program will not cause a memory access violation.

Typed lambda calculi are closely related to mathematical logic and proof theory via the Curry–Howard isomorphism and they can be considered as the internal language of classes of categories, e.g., the simply typed lambda calculus is the language of a Cartesian closed category (CCC).

Reduction strategies

Whether a term is normalising or not, and how much work needs to be done in normalising it if it is, depends to a large extent on the reduction strategy used. Common lambda calculus reduction strategies include:[29][30][31]

Normal order
: The leftmost outermost redex is reduced first. That is, whenever possible, arguments are substituted into the body of an abstraction before the arguments are reduced. If a term has a beta-normal form, normal order reduction will always reach that normal form.

Applicative order
: The leftmost innermost redex is reduced first. As a consequence, a function's arguments are always reduced before they are substituted into the function. Unlike normal order reduction, applicative order reduction may fail to find the beta-normal form of an expression, even if such a normal form exists. For example, the term

$(\ \lambda x.\, y\ \ (\lambda z.\, (zz)\ \lambda z.\, (zz))\ )$ is reduced to itself by applicative order, while normal order reduces it to its beta-normal form y.

Full β-reductions

Any redex can be reduced at any time. This means essentially the lack of any particular reduction strategy—with regard to reducibility, "all bets are off".

Weak reduction strategies do not reduce under lambda abstractions:

Call by value

Like applicative order, but no reductions are performed inside abstractions. This is similar to the evaluation order of strict languages like C: the arguments to a function are evaluated before calling the function, and function bodies are not even partially evaluated until the arguments are substituted in.

Call by name

Like normal order, but no reductions are performed inside abstractions. For example, $\lambda x.\, (\lambda y.\, y)\, x$ is in normal form according to this strategy, although it contains the redex $(\lambda y.\, y)\, x$.

Strategies with sharing reduce computations that are "the same" in parallel:

Optimal reduction

As normal order, but computations that have the same label are reduced simultaneously.

Call by need

As call by name (hence weak), but function applications that would duplicate terms instead name the argument. The argument may be evaluated "when needed", at which point the name binding is updated with the reduced value. This can save time compared to normal order evaluation.

Computability

There is no algorithm that takes as input any two lambda expressions and outputs TRUE or FALSE depending on whether one expression reduces to the other.[15] More precisely, no computable function can decide the question. This was historically the first problem for which undecidability could be proven. As usual for such a proof, *computable* means computable by any model of computation that is Turing complete. In fact computability can itself be defined via the lambda calculus: a function F: $\mathbf{N} \to \mathbf{N}$ of natural numbers is a computable function if and only if there exists a lambda expression f such that for every pair of x, y in $\mathbf{N}$, $F(x)=y$ if and only if $f\ x =_\beta y$, where x and y are the Church numerals corresponding to x and y, respectively and $=_\beta$ meaning equivalence with β-reduction. See the Church–Turing thesis for other approaches to defining computability and their equivalence.

Church's proof of uncomputability first reduces the problem to determining whether a given lambda expression has a normal form. Then he assumes that this predicate is computable, and can hence be expressed in lambda calculus. Building on earlier work by Kleene and constructing a Gödel numbering for lambda expressions, he constructs a lambda expression e that closely follows the proof of Gödel's first incompleteness theorem. If e is applied to its own Gödel number, a contradiction results.

Complexity

The notion of computational complexity for the lambda calculus is a bit tricky, because the cost of a β-reduction may vary depending on how it is implemented.[32] To be precise, one must somehow find the location of all of the occurrences of the bound variable V in the expression E, implying a time cost, or one must keep track of the locations of free variables in some way, implying a space cost. A naïve search for the locations of V in E is $O(n)$ in the length n of E. Director strings were an early approach that traded this time cost for a quadratic space usage.[33] More generally this has led to the study of systems that use explicit substitution.

In 2014, it was shown that the number of β-reduction steps taken by normal order reduction to reduce a term is a *reasonable* time cost model, that is, the reduction can be simulated on a Turing machine in time polynomially proportional to the number of steps.[34] This was a long-standing open problem, due to *size explosion*, the existence of lambda terms which grow exponentially in size for each β-reduction. The result gets around this by working with a compact shared representation. The result makes clear that the amount of space needed to evaluate a lambda term is not proportional to the size of the term during reduction. It is not currently known what a good measure of space complexity would be.[35]

An unreasonable model does not necessarily mean inefficient. Optimal reduction reduces all computations with the same label in one step, avoiding duplicated work, but the number of parallel β-reduction steps to reduce a given term to normal form is approximately linear in the size of the term. This is far too small to be a reasonable cost measure, as any Turing machine may be encoded in the lambda calculus in size linearly proportional to the size of the Turing machine. The true cost of reducing lambda terms is not due to β-reduction per se but rather the handling of the duplication of redexes during β-reduction.[36] It is not known if optimal reduction implementations are reasonable when measured with respect to a reasonable cost model such as the number of leftmost-outermost steps to normal form, but it has been shown for fragments of the lambda calculus that the optimal reduction algorithm is efficient and has at most a quadratic overhead compared to leftmost-outermost.[35] In addition the BOHM prototype implementation of optimal reduction outperformed both Caml Light and Haskell on pure lambda terms.[36]

Lambda calculus and programming languages

As pointed out by Peter Landin's 1965 paper "*A Correspondence between ALGOL 60 and Church's Lambda-notation*",[37] sequential procedural programming languages can be understood in terms of the lambda calculus, which provides the basic mechanisms for procedural abstraction and procedure (subprogram) application.

Anonymous functions

For example, in Python the "square" function can be expressed as a lambda expression as follows:

```
(lambda x: x**2)
```

The above example is an expression that evaluates to a first-class function. The symbol `lambda` creates an anonymous function, given a list of parameter names—just the single argument `x`, in this case—and an expression that is evaluated as the body of the function, `x**2`. Anonymous functions are sometimes called *lambda expressions.*

Pascal and many other imperative languages have long supported passing subprograms as arguments to other subprograms through the mechanism of function pointers. However, function pointers are an insufficient condition for functions to be first class datatypes, because a function is a first class datatype if and only if new instances of the function can be created at runtime. Such runtime creation of functions is supported in Smalltalk, JavaScript, Wolfram Language, and more recently in Scala, Eiffel (as agents), C# (as delegates) and C++11, among others.

Parallelism and concurrency

The Church–Rosser property of the lambda calculus means that evaluation (β-reduction) can be carried out in *any order*, even in parallel. This means that various nondeterministic evaluation strategies are relevant. However, the lambda calculus does not offer any explicit constructs for parallelism. One can add constructs such as futures to the lambda calculus. Other process calculi have been developed for describing communication and concurrency.

Semantics

The fact that lambda calculus terms act as functions on other lambda calculus terms, and even on themselves, led to questions about the semantics of the lambda calculus. Could a sensible meaning be assigned to lambda calculus terms? The natural semantics was to find a set D isomorphic to the function space $D \to D$, of functions on itself. However, no nontrivial such D can exist, by cardinality constraints because the set of all functions from D to D has greater cardinality than D, unless D is a singleton set.

In the 1970s, Dana Scott showed that if only continuous functions were considered, a set or domain D with the required property could be found, thus providing a model for the lambda calculus.[38]

This work also formed the basis for the denotational semantics of programming languages.

Variations and extensions

These extensions are in the lambda cube:

- Typed lambda calculus – Lambda calculus with typed variables (and functions)
- System F – A typed lambda calculus with type-variables
- Calculus of constructions – A typed lambda calculus with types as first-class values

These formal systems are extensions of lambda calculus that are not in the lambda cube:

- Binary lambda calculus – A version of lambda calculus with binary input/output (I/O), a binary encoding of terms, and a designated universal machine.
- Lambda-mu calculus – An extension of the lambda calculus for treating classical logic

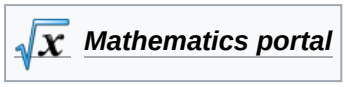
These formal systems are variations of lambda calculus:

- Kappa calculus – A first-order analogue of lambda calculus

These formal systems are related to lambda calculus:

- Combinatory logic – A notation for mathematical logic without variables
- SKI combinator calculus – A computational system based on the **S**, **K** and **I** combinators, equivalent to lambda calculus, but reducible without variable substitutions

See also



- Applicative computing systems – Treatment of objects in the style of the lambda calculus
- Cartesian closed category – A setting for lambda calculus in category theory
- Categorical abstract machine – A model of computation applicable to lambda calculus
- Clojure, programming language
- Curry–Howard isomorphism – The formal correspondence between programs and proofs
- De Bruijn index – notation disambiguating alpha conversions
- De Bruijn notation – notation using postfix modification functions
- Domain theory – Study of certain posets giving denotational semantics for lambda calculus
- Evaluation strategy – Rules for the evaluation of expressions in programming languages
- Explicit substitution – The theory of substitution, as used in β-reduction
- Harrop formula – A kind of constructive logical formula such that proofs are lambda terms
- Interaction nets
- Kleene–Rosser paradox – A demonstration that some form of lambda calculus is inconsistent
- Knights of the Lambda Calculus – A semi-fictional organization of LISP and Scheme hackers
- Krivine machine – An abstract machine to interpret call-by-name in lambda calculus
- Lambda calculus definition – Formal definition of the lambda calculus.
- Let expression – An expression closely related to an abstraction.
- Minimalism (computing)
- Rewriting – Transformation of formulæ in formal systems
- SECD machine – A virtual machine designed for the lambda calculus
- Scott–Curry theorem – A theorem about sets of lambda terms
- *To Mock a Mockingbird* – An introduction to combinatory logic
- Universal Turing machine – A formal computing machine equivalent to lambda calculus
- Unlambda – A functional esoteric programming language based on combinatory logic

Further reading

- Abelson, Harold & Gerald Jay Sussman. Structure and Interpretation of Computer Programs. The MIT Press. ISBN 0-262-51087-1.
- Barendregt, Hendrik Pieter *Introduction to Lambda Calculus* (http://www.cse.chalmers.se/research/group/logic/TypesSS05/Extra/geuvers.pdf).
- Barendregt, Hendrik Pieter, The Impact of the Lambda Calculus in Logic and Computer Science (https://www.jstor.org/stable/421013). The Bulletin of Symbolic Logic, Volume 3,

Number 2, June 1997.
- Barendregt, Hendrik Pieter, *The Type Free Lambda Calculus* pp1091–1132 of *Handbook of Mathematical Logic*, North-Holland (1977) ISBN 0-7204-2285-X
- Cardone, Felice and Hindley, J. Roger, 2006. History of Lambda-calculus and Combinatory Logic (http://www.users.waitrose.com/~hindley/SomePapers_PDFs/2006CarHin,HistlamRp.pdf) Archived (https://web.archive.org/web/20210506154120/http://www.users.waitrose.com/~hindley/SomePapers_PDFs/2006CarHin,HistlamRp.pdf) 2021-05-06 at the Wayback Machine. In Gabbay and Woods (eds.), *Handbook of the History of Logic*, vol. 5. Elsevier.
- Church, Alonzo, *An unsolvable problem of elementary number theory*, American Journal of Mathematics, 58 (1936), pp. 345–363. This paper contains the proof that the equivalence of lambda expressions is in general not decidable.
- Church, Alonzo (1941). *The Calculi of Lambda-Conversion* (https://archive.org/details/AnnalsOfMathematicalStudies6ChurchAlonzoTheCalculiOfLambdaConversionPrincetonUniversityPress1941). Princeton: Princeton University Press. Retrieved 2020-04-14. (ISBN 978-0-691-08394-0)
- Frink Jr., Orrin (1944). "Review: *The Calculi of Lambda-Conversion* by Alonzo Church" (https://www.ams.org/bull/1944-50-03/S0002-9904-1944-08090-7/S0002-9904-1944-08090-7.pdf) (PDF). *Bulletin of the American Mathematical Society*. **50** (3): 169–172. doi:10.1090/s0002-9904-1944-08090-7 (https://doi.org/10.1090%2Fs0002-9904-1944-08090-7).
- Kleene, Stephen, *A theory of positive integers in formal logic*, American Journal of Mathematics, 57 (1935), pp. 153–173 and 219–244. Contains the lambda calculus definitions of several familiar functions.
- Landin, Peter, *A Correspondence Between ALGOL 60 and Church's Lambda-Notation*, Communications of the ACM, vol. 8, no. 2 (1965), pages 89–101. Available from the ACM site (http://portal.acm.org/citation.cfm?id=363749&coll=portal&dl=ACM). A classic paper highlighting the importance of lambda calculus as a basis for programming languages.
- Larson, Jim, *An Introduction to Lambda Calculus and Scheme* (https://web.archive.org/web/20011206080336/http://www.jetcafe.org/~jim/lambda.html). A gentle introduction for programmers.
- Michaelson, Greg (10 April 2013). *An Introduction to Functional Programming Through Lambda Calculus*. Courier Corporation. ISBN 978-0-486-28029-5.[39]
- Schalk, A. and Simmons, H. (2005) *An introduction to λ-calculi and arithmetic with a decent selection of exercises (https://web.archive.org/web/20080307014129/http://www.cs.man.ac.uk/~hsimmons/BOOKS/lcalculus.pdf)*. Notes for a course in the Mathematical Logic MSc at Manchester University.
- de Queiroz, Ruy J.G.B. (2008). "On Reduction Rules, Meaning-as-Use and Proof-Theoretic Semantics". *Studia Logica*. **90** (2): 211–247. doi:10.1007/s11225-008-9150-5 (https://doi.org/10.1007%2Fs11225-008-9150-5). S2CID 11321602 (https://api.semanticscholar.org/CorpusID:11321602). A paper giving a formal underpinning to the idea of 'meaning-is-use' which, even if based on proofs, it is different from proof-theoretic semantics as in the Dummett–Prawitz tradition since it takes reduction as the rules giving meaning.
- Hankin, Chris, *An Introduction to Lambda Calculi for Computer Scientists,* ISBN 0954300653

Monographs/textbooks for graduate students

- Sørensen, Morten Heine and Urzyczyn, Paweł (2006), *Lectures on the Curry–Howard isomorphism*, Elsevier, ISBN 0-444-52077-5 is a recent monograph that covers the main topics of lambda calculus from the type-free variety, to most typed lambda calculi, including more recent developments like pure type systems and the lambda cube. It does not cover subtyping extensions.

- Pierce, Benjamin (2002), *Types and Programming Languages*, MIT Press, ISBN 0-262-16209-1 covers lambda calculi from a practical type system perspective; some topics like dependent types are only mentioned, but subtyping is an important topic.

Documents

- *A Short Introduction to the Lambda Calculus (http://www.cs.bham.ac.uk/~axj/pub/papers/lambda-calculus.pdf)*-(PDF) by Achim Jung
- *A timeline of lambda calculus (http://turing100.acm.org/lambda_calculus_timeline.pdf)*-(PDF) by Dana Scott
- *A Tutorial Introduction to the Lambda Calculus (http://www.inf.fu-berlin.de/inst/ag-ki/rojas_home/documents/tutorials/lambda.pdf)*-(PDF) by Raúl Rojas
- *Lecture Notes on the Lambda Calculus (http://www.mscs.dal.ca/~selinger/papers/#lambdanotes)*-(PDF) by Peter Selinger
- *Graphic lambda calculus* (https://web.archive.org/web/20140202195546/http://imar.ro/~mbuliga/graphic_revised.pdf) by Marius Buliga
- *Lambda Calculus as a Workflow Model* (https://web.archive.org/web/20160729210437/http://cs.adelaide.edu.au/~pmk/publications/wage2008.pdf) by Peter Kelly, Paul Coddington, and Andrew Wendelborn; mentions graph reduction as a common means of evaluating lambda expressions and discusses the applicability of lambda calculus for distributed computing (due to the Church–Rosser property, which enables parallel graph reduction for lambda expressions).

Notes

a. "where M[x := N] denotes the substitution of N for every occurrence of x in M".[1]:7 Also denoted M[N/x], "the substitution of N for x in M".[2]

b. Barendregt, Barendsen (2000) call this rule *axiom β*

c. For a full history, see Cardone and Hindley's "History of Lambda-calculus and Combinatory Logic" (2006).

d. $\mapsto$ is pronounced "maps to".

e. $(\lambda f.M)\ N$ can be pronounced "let f be N in M".

f. Ariola and Blom[22] employ 1) axioms for a representational calculus using *well-formed cyclic lambda graphs* extended with `letrec`, to detect possibly infinite unwinding trees; 2) the representational calculus with β-reduction of scoped lambda graphs constitute Ariola/Blom's cyclic extension of lambda calculus; 3) Ariola/Blom reason about strict languages using § call-by-value, and compare to Moggi's calculus, and to Hasegawa's calculus. Conclusions on p. 111.[22]

References

Some parts of this article are based on material from FOLDOC, used with permission.

1. Barendregt, Henk; Barendsen, Erik (March 2000), *Introduction to Lambda Calculus* (https://ftp.science.ru.nl/CSI/CompMath.Found/lambda.pdf) (PDF)
2. explicit substitution (https://ncatlab.org/nlab/show/explicit+substitution) at the *n*Lab
3. de Queiroz, Ruy J. G. B. (1988). "A Proof-Theoretic Account of Programming and the Role of Reduction Rules". *Dialectica*. **42** (4): 265–282. doi:10.1111/j.1746-8361.1988.tb00919.x (https://doi.org/10.1111%2Fj.1746-8361.1988.tb00919.x).

4. Turing, Alan M. (December 1937). "Computability and λ-Definability". *The Journal of Symbolic Logic*. **2** (4): 153–163. doi:10.2307/2268280 (https://doi.org/10.2307%2F2268280). JSTOR 2268280 (https://www.jstor.org/stable/2268280). S2CID 2317046 (https://api.semanticscholar.org/CorpusID:2317046).
5. Tait, W. W. (August 1967). "Intensional interpretations of functionals of finite type I" (https://doi.org/10.2307/2271658). *The Journal of Symbolic Logic*. **32** (2): 198–212. doi:10.2307/2271658 (https://doi.org/10.2307%2F2271658). ISSN 0022-4812 (https://search.worldcat.org/issn/0022-4812). JSTOR 2271658 (https://www.jstor.org/stable/2271658). S2CID 9569863 (https://api.semanticscholar.org/CorpusID:9569863).
6. Coquand, Thierry (8 February 2006). Zalta, Edward N. (ed.). "Type Theory" (http://plato.stanford.edu/archives/sum2013/entries/type-theory/). *The Stanford Encyclopedia of Philosophy* (Summer 2013 ed.). Retrieved November 17, 2020.
7. Moortgat, Michael (1988). *Categorial Investigations: Logical and Linguistic Aspects of the Lambek Calculus* (https://books.google.com/books?id=9CdFE9X_FCoC). Foris Publications. ISBN 9789067653879.
8. Bunt, Harry; Muskens, Reinhard, eds. (2008). *Computing Meaning* (https://books.google.com/books?id=nyFa5ngYThMC). Springer. ISBN 978-1-4020-5957-5.
9. Mitchell, John C. (2003). *Concepts in Programming Languages* (https://books.google.com/books?id=7Uh8XGfJbEIC&pg=PA57). Cambridge University Press. p. 57. ISBN 978-0-521-78098-8..
10. Chacón Sartori, Camilo (2023-12-05). *Introduction to Lambda Calculus using Racket* (https://web.archive.org/web/20231207230752/https://www.researchgate.net/publication/376232150_Introduction_to_Lambda_Calculus_using_Racket) (Technical report). Archived from the original (https://www.researchgate.net/publication/376232150) on 2023-12-07.
11. Pierce, Benjamin C. *Basic Category Theory for Computer Scientists*. p. 53.
12. Church, Alonzo (1932). "A set of postulates for the foundation of logic". *Annals of Mathematics*. Series 2. **33** (2): 346–366. doi:10.2307/1968337 (https://doi.org/10.2307%2F1968337). JSTOR 1968337 (https://www.jstor.org/stable/1968337).
13. Kleene, Stephen C.; Rosser, J. B. (July 1935). "The Inconsistency of Certain Formal Logics". *The Annals of Mathematics*. **36** (3): 630. doi:10.2307/1968646 (https://doi.org/10.2307%2F1968646). JSTOR 1968646 (https://www.jstor.org/stable/1968646).
14. Church, Alonzo (December 1942). "Review of Haskell B. Curry, *The Inconsistency of Certain Formal Logics*". *The Journal of Symbolic Logic*. **7** (4): 170–171. doi:10.2307/2268117 (https://doi.org/10.2307%2F2268117). JSTOR 2268117 (https://www.jstor.org/stable/2268117).
15. Church, Alonzo (1936). "An unsolvable problem of elementary number theory". *American Journal of Mathematics*. **58** (2): 345–363. doi:10.2307/2371045 (https://doi.org/10.2307%2F2371045). JSTOR 2371045 (https://www.jstor.org/stable/2371045).
16. Church, Alonzo (1940). "A Formulation of the Simple Theory of Types". *Journal of Symbolic Logic*. **5** (2): 56–68. doi:10.2307/2266170 (https://doi.org/10.2307%2F2266170). JSTOR 2266170 (https://www.jstor.org/stable/2266170). S2CID 15889861 (https://api.semanticscholar.org/CorpusID:15889861).
17. Partee, B. B. H.; ter Meulen, A.; Wall, R. E. (1990). *Mathematical Methods in Linguistics* (https://books.google.com/books?id=qV7TUuaYcUIC&pg=PA317). Springer. ISBN 9789027722454. Retrieved 29 Dec 2016.
18. Alama, Jesse. Zalta, Edward N. (ed.). "The Lambda Calculus" (http://plato.stanford.edu/entries/lambda-calculus/). *The Stanford Encyclopedia of Philosophy* (Summer 2013 ed.). Retrieved November 17, 2020.

19. Dana Scott, "Looking Backward; Looking Forward (https://www.youtube.com/embed/uS9InrmPIoc)", Invited Talk at the Workshop in honour of Dana Scott's 85th birthday and 50 years of domain theory, 7–8 July, FLoC 2018 (talk 7 July 2018). The relevant passage begins at 32:50 (https://www.youtube.com/embed/uS9InrmPIoc?start=1970). (See also this extract of a May 2016 talk (https://www.youtube.com/watch?time_continue=1&v=juXwu0Nqc3I) at the University of Birmingham, UK.)
20. Felleisen, Matthias; Flatt, Matthew (2006), *Programming Languages and Lambda Calculi* (https://web.archive.org/web/20090205113235/http://www.cs.utah.edu/plt/publications/pllc.pdf) (PDF), p. 26, archived from the original (http://www.cs.utah.edu/plt/publications/pllc.pdf) (PDF) on 2009-02-05; A note (accessed 2017) at the original location suggests that the authors consider the work originally referenced to have been superseded by a book.
21. Selinger, Peter (2008), *Lecture Notes on the Lambda Calculus* (http://www.mathstat.dal.ca/~selinger/papers/lambdanotes.pdf) (PDF), vol. 0804, Department of Mathematics and Statistics, University of Ottawa, p. 9, arXiv:0804.3434 (https://arxiv.org/abs/0804.3434), Bibcode:2008arXiv0804.3434S (https://ui.adsabs.harvard.edu/abs/2008arXiv0804.3434S)
22. Zena M. Ariola and Stefan Blom, *Proc. TACS '94* Sendai, Japan 1997 (1997) Cyclic lambda calculi (http://ix.cs.uoregon.edu/~ariola/cycles.pdf) 114 pages.
23. Bene, Adam (17 August 2017). "Fixed-Point Combinators in JavaScript" (https://benestudio.co/fixed-point-combinators-in-javascript/). *Bene Studio*. Medium. Retrieved 2 August 2020.
24. "CS 6110 S17 Lecture 5. Recursion and Fixed-Point Combinators" (https://www.cs.cornell.edu/courses/cs6110/2017sp/lectures/lec05.pdf) (PDF). *Cornell University*. 4.1 A CBV Fixed-Point Combinator.
25. Ker, Andrew D. "Lambda Calculus and Types" (https://www.cs.ox.ac.uk/andrew.ker/docs/lambdacalculus-lecture-notes-ht2009.pdf#page=18) (PDF). p. 6. Retrieved 14 January 2022.
26. Dezani-Ciancaglini, Mariangiola; Ghilezan, Silvia (2014). "Preciseness of Subtyping on Intersection and Union Types" (http://www.di.unito.it/~dezani/papers/dg14.pdf#page=3) (PDF). *Rewriting and Typed Lambda Calculi*. Lecture Notes in Computer Science. Vol. 8560. p. 196. doi:10.1007/978-3-319-08918-8_14 (https://doi.org/10.1007%2F978-3-319-08918-8_14). hdl:2318/149874 (https://hdl.handle.net/2318%2F149874). ISBN 978-3-319-08917-1. Retrieved 14 January 2022.
27. Forster, Yannick; Smolka, Gert (August 2019). "Call-by-Value Lambda Calculus as a Model of Computation in Coq" (https://www.ps.uni-saarland.de/Publications/documents/ForsterSmolka_2018_Computability-JAR.pdf#page=4) (PDF). *Journal of Automated Reasoning*. **63** (2): 393–413. doi:10.1007/s10817-018-9484-2 (https://doi.org/10.1007%2Fs10817-018-9484-2). S2CID 53087112 (https://api.semanticscholar.org/CorpusID:53087112). Retrieved 14 January 2022.
28. Types and Programming Languages, p. 273, Benjamin C. Pierce
29. Pierce, Benjamin C. (2002). *Types and Programming Languages* (https://books.google.com/books?id=ti6zoAC9Ph8C&pg=PA56). MIT Press. p. 56. ISBN 0-262-16209-1.
30. Sestoft, Peter (2002). "Demonstrating Lambda Calculus Reduction" (http://itu.dk/people/sestoft/papers/sestoft-lamreduce.pdf) (PDF). *The Essence of Computation*. Lecture Notes in Computer Science. Vol. 2566. pp. 420–435. doi:10.1007/3-540-36377-7_19 (https://doi.org/10.1007%2F3-540-36377-7_19). ISBN 978-3-540-00326-7. Retrieved 22 August 2022.
31. Biernacka, Małgorzata; Charatonik, Witold; Drab, Tomasz (2022). Andronick, June; de Moura, Leonardo (eds.). *The Zoo of Lambda-Calculus Reduction Strategies, and Coq* (https://drops.dagstuhl.de/opus/volltexte/2022/16716/pdf/LIPIcs-ITP-2022-7.pdf) (PDF). Leibniz International Proceedings in Informatics (LIPIcs). Vol. 237. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. pp. 7:1–7:19. doi:10.4230/LIPIcs.ITP.2022.7 (https://doi.org/10.4230%2FLIPIcs.ITP.2022.7). ISBN 978-3-95977-252-5. Retrieved 22 August 2022.

32. Frandsen, Gudmund Skovbjerg; Sturtivant, Carl (26 August 1991). "What is an efficient implementation of the λ-calculus?" (https://scholar.archive.org/work/i7pfbnat3vdwfgalk5l6pkvbze). *Functional Programming Languages and Computer Architecture: 5th ACM Conference. Cambridge, MA, USA, August 26-30, 1991. Proceedings*. Lecture Notes in Computer Science. Vol. 523. Springer-Verlag. pp. 289–312. CiteSeerX 10.1.1.139.6913 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.139.6913). doi:10.1007/3540543961_14 (https://doi.org/10.1007%2F3540543961_14). ISBN 9783540543961.
33. Sinot, F.-R. (2005). "Director Strings Revisited: A Generic Approach to the Efficient Representation of Free Variables in Higher-order Rewriting" (http://www.lsv.fr/Publis/PAPERS/PDF/sinot-jlc05.pdf) (PDF). *Journal of Logic and Computation*. **15** (2): 201–218. doi:10.1093/logcom/exi010 (https://doi.org/10.1093%2Flogcom%2Fexi010).
34. Accattoli, Beniamino; Dal Lago, Ugo (14 July 2014). "Beta reduction is invariant, indeed". *Proceedings of the Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic (CSL) and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. pp. 1–10. arXiv:1601.01233 (https://arxiv.org/abs/1601.01233). doi:10.1145/2603088.2603105 (https://doi.org/10.1145%2F2603088.2603105). ISBN 9781450328869. S2CID 11485010 (https://api.semanticscholar.org/CorpusID:11485010).
35. Accattoli, Beniamino (October 2018). "(In)Efficiency and Reasonable Cost Models" (https://doi.org/10.1016%2Fj.entcs.2018.10.003). *Electronic Notes in Theoretical Computer Science*. **338**: 23–43. doi:10.1016/j.entcs.2018.10.003 (https://doi.org/10.1016%2Fj.entcs.2018.10.003).
36. Asperti, Andrea (16 Jan 2017). "About the efficient reduction of lambda terms". arXiv:1701.04240v1 (https://arxiv.org/abs/1701.04240v1) [cs.LO (https://arxiv.org/archive/cs.LO)].
37. Landin, P. J. (1965). "A Correspondence between ALGOL 60 and Church's Lambda-notation" (https://doi.org/10.1145%2F363744.363749). *Communications of the ACM*. **8** (2): 89–101. doi:10.1145/363744.363749 (https://doi.org/10.1145%2F363744.363749). S2CID 6505810 (https://api.semanticscholar.org/CorpusID:6505810).
38. Scott, Dana (1993). "A type-theoretical alternative to ISWIM, CUCH, OWHY" (https://www.cs.cmu.edu/~crary/819-f09/Scott93.pdf) (PDF). *Theoretical Computer Science*. **121** (1–2): 411–440. doi:10.1016/0304-3975(93)90095-B (https://doi.org/10.1016%2F0304-3975%2893%2990095-B). Retrieved 2022-12-01. Written 1969, widely circulated as an unpublished manuscript.
39. "Greg Michaelson's Homepage" (http://www.macs.hw.ac.uk/~greg/). *Mathematical and Computer Sciences*. Riccarton, Edinburgh: Heriot-Watt University. Retrieved 6 November 2022.

External links

- Graham Hutton, Lambda Calculus (https://www.youtube.com/watch?v=eis11j_iGMs), a short (12 minutes) Computerphile video on the Lambda Calculus
- Helmut Brandl, *Step by Step Introduction to Lambda Calculus (https://hbr.github.io/Lambda-Calculus/)*
- "Lambda-calculus" (https://www.encyclopediaofmath.org/index.php?title=Lambda-calculus), *Encyclopedia of Mathematics*, EMS Press, 2001 [1994]
- David C. Keenan, *To Dissect a Mockingbird: A Graphical Notation for the Lambda Calculus with Animated Reduction (http://dkeenan.com/Lambda/)*
- L. Allison, *Some executable λ-calculus examples (http://www.allisons.org/ll/FP/Lambda/Examples/)*

- Georg P. Loczewski, *The Lambda Calculus and A++* (http://www.lambda-bound.com/book/lambdacalc/lcalconl.html)
- Bret Victor, *Alligator Eggs: A Puzzle Game Based on Lambda Calculus (http://worrydream.com/AlligatorEggs/)*
- *Lambda Calculus (http://www.safalra.com/science/lambda-calculus/) Archived (https://web.archive.org/web/20121014180848/http://safalra.com/science/lambda-calculus/) 2012-10-14 at the Wayback Machine* on Safalra's Website (http://www.safalra.com/) Archived (https://web.archive.org/web/20210502051230/https://safalra.com/) 2021-05-02 at the Wayback Machine
- LCI Lambda Interpreter (https://chatziko.github.io/lci/) a simple yet powerful pure calculus interpreter
- Lambda Calculus links on Lambda-the-Ultimate (http://lambda-the-ultimate.org/classic/lc.html)
- Mike Thyer, Lambda Animator (https://web.archive.org/web/20070713173324/http://thyer.name/lambda-animator/), a graphical Java applet demonstrating alternative reduction strategies.
- Implementing the Lambda calculus (http://matt.might.net/articles/c++-template-meta-programming-with-lambda-calculus/) using C++ Templates
- Shane Steinert-Threlkeld, "Lambda Calculi" (https://web.archive.org/web/20141216225504/http://www.iep.utm.edu/lambda-calculi/), *Internet Encyclopedia of Philosophy*
- Anton Salikhmetov, *Macro Lambda Calculus* (https://codedot.github.io/lambda/)
